

Academy Backend/Frontend/QE Virtual MTY

Proyecto: 01-Core-Avanzado

Desarrollador: Sergio Servando Prado Lozano

Encargado: Miguel Ángel Rugerio Flores

Lugar: San Pedro de las Colonias Coahuila

Fecha: 29/08/2026

Concepto 1.1: Threading y Concurrency

Antes, los procesadores tenían un solo núcleo y los programas hacían una sola cosa a la vez. Cuando las computadoras se volvieron más potentes y multiproceso, los lenguajes como Java necesitaron una forma de aprovechar ese potencial para no desperdiciarlo: de ahí nacen los hilos (Threads). En el contexto del cine qué es el pequeño programa sencillito que hice en base a lo investigado y aprendido, es que, si el programa corre todo en el hilo principal el main thread, es como tener a un solo cajero; la fila avanza muy lenta porque el sistema no puede atender al cliente 2 hasta que terminé con el cliente 1. Los hilos resuelven este cuello de botella así es como en los juegos cuando muchos procesos pasan a la vez y tu gráfica que le envía miles de datos simultáneos si el poder de proceso por parte del procesador es deficiente se genera el cuello de botella, en caso de los Threads se resuelve permitiendo que el sistema operativo divida el trabajo y abra "varias cajas" al mismo tiempo. Así logramos que el programa haga varias tareas de forma paralela sin quedarse pasmado.

Todo está en un solo archivo: TaquillaCine.java.

Ahí usé un `ExecutorService` llamado cajeros para crear un pool fijo de 5 hilos. En lugar de crear hilos manualmente uno por uno a lo loco, el `ExecutorService` administra estos 5 cajeros virtuales de forma eficiente, asignándoles trabajo las ventas al mismo tiempo mediante el comando `cajeros.submit()`.

```

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;

public class TaquillaCine {
    // Esta variable es la que fallará porque es un número normal.
    // Si varios hilos o cajeros cómo también se le conocen intentan sumarle al mismo tiempo, se van a
    // estar estorbando unos a otros.
    private static int boletosSinProteccion = 0;

    // Esta variable es especial o mejor dicho es el AtomicInteger.
    // Es la que le dice a los hilos a tener un orden por así decirlo y sumar de uno en uno para no
    // perder la cuenta.
    private static AtomicInteger boletosProtegidos = new AtomicInteger(0);

    public static void main(String[] args) throws InterruptedException {
        System.out.println("Abriendo la taquilla del cine...");

        // Contratamos a 5 cajeros para que atiendan al mismo tiempo que se le conoce también como
        // pool de 5 hilos
        ExecutorService cajeros = Executors.newFixedThreadPool(5);

        // Llegan 10,000 personas a comprar boletos de golpe
        for (int i = 0; i < 10000; i++) {
            cajeros.submit(() -> {
                // Los cajeros o hilos intentan sumar en la cuenta sin protección aquí es donde se
                // desordenan y empieza la pérdidas de ventas.
                boletosSinProteccion++;

                // En este los cajeros o hilos suman en la cuenta protegida aquí si llevan un orden
                // por eso siempre darán un buen resultado.
                boletosProtegidos.incrementAndGet();
            });
        }

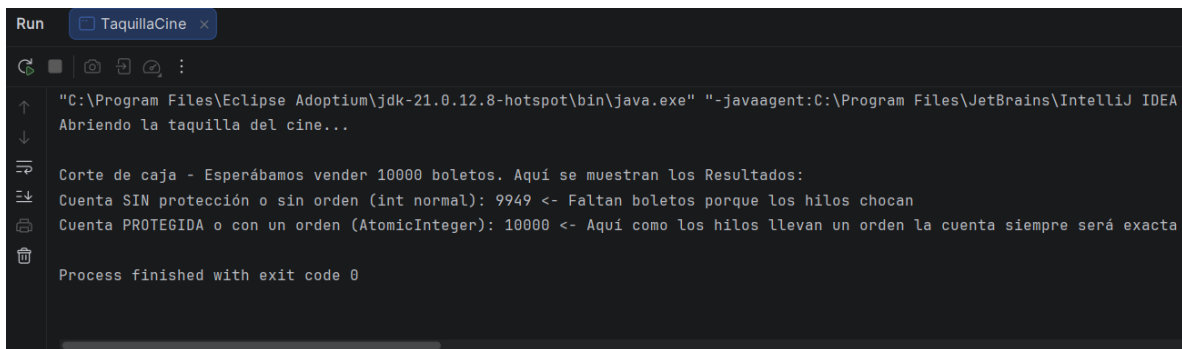
        // Aquí decimos que ya no recibiremos más clientes (shutdown) y esperamos a que los cajeros
        // terminen su fila (await)
        cajeros.shutdown();
        cajeros.awaitTermination(1, TimeUnit.MINUTES);

        // Revisamos el corte de caja al final del día
        System.out.println("\nCorte de caja - Esperábamos vender 10000 boletos. Aquí se muestran los
        Resultados:");
        System.out.println("Cuenta SIN protección o sin orden (int normal): " + boletosSinProteccion +
        " <- Faltan boletos porque los hilos chocan");
        System.out.println("Cuenta PROTEGIDA o con un orden (AtomicInteger): " +
        boletosProtegidos.get() + " <- Aquí como los hilos llevan un orden la cuenta siempre será exacta");
    }
}

```

El problema de usar hilos es que todos comparten la misma memoria RAM. Si pones a varios hilos a modificar el mismo dato sin ponerles reglas, chocan. En mi código lo demuestro con la variable `int boletosSinProteccion`. Si 5 hilos intentan sumarle "+1" exactamente en el mismo milisegundo, la computadora no hace 5 sumas ordenadas.

Lo que pasa por debajo es que los 5 hilos leen el número actual al mismo tiempo por ejemplo 100, los 5 hacen la suma 101, y los 5 escriben 101 en la memoria a la vez. Se acaban de perder 4 venta, a esto se le conoce como condición de carrera lo cual puede generar corrupción de datos. Para que no se rompa, se solucionó usando la clase `AtomicInteger` en la variable `boletosProtegidos`. Esta clase resuelve la concurrencia haciendo que la suma sea indivisible. Funciona como un candado a nivel hardware: obliga a los hilos a formarse una fracción de milisegundo para que uno termine de sumar y guardar antes de que el siguiente pueda leer el valor. Así, la cuenta es perfecta y no se pierde ni un boleto. Aquí se muestra la ejecución del programa.



```
Run TaquillaCine x
"C:\Program Files\Eclipse Adoptium\jdk-21.0.12.8-hotspot\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA
Abriendo la taquilla del cine...

Corte de caja - Esperábamos vender 10000 boletos. Aquí se muestran los Resultados:
Cuenta SIN protección o sin orden (int normal): 9949 <- Faltan boletos porque los hilos chocan
Cuenta PROTEGIDA o con un orden (AtomicInteger): 10000 <- Aquí como los hilos llevan un orden la cuenta siempre será exacta

Process finished with exit code 0
```

Concepto 1.2: Manejo de Archivos y Streams

Básicamente, todo programa pierde sus datos cuando se cierra porque viven en la memoria RAM. El manejo de archivos nos ayuda a guardar la información en el disco duro para que sobreviva. El problema aquí es que si intentamos cargar un archivo muy pesado de golpe, la memoria RAM se satura y el programa se cae. Esto se resuelve usando Streams o los flujos de datos. Un stream funciona, así como lo va pasando y procesando poco a poco con buffers. Además, al usar la estructura try-with-resources, resolvemos el problema de dejar archivos abiertos por accidente. El mejor ejemplo de esto está en el paquete com.curso.streams.v1, específicamente en la clase PrincipalPath03.java.

```
package src.com.curso.streams.v1;

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;

public class PrincipalPath03 {

    public static void main(String[] args) throws IOException {

        String currentDir = System.getProperty("user.dir");
        Path pathInput = Paths.get(currentDir, "data", "origen.txt");

        Path pathOutput = Paths.get(currentDir, "data", "destino.txt");
        copyPath(pathInput, pathOutput);

        System.out.println("Listo!!!");

    }

    static private void copyPath(Path input, Path output) throws IOException
    {

        try (BufferedReader reader = Files.newBufferedReader(input);
            BufferedWriter writer = Files.newBufferedWriter(output)) {

            String line = null;

            while ((line = reader.readLine()) != null) {
                writer.write(line);
                writer.newLine();
            }

        }

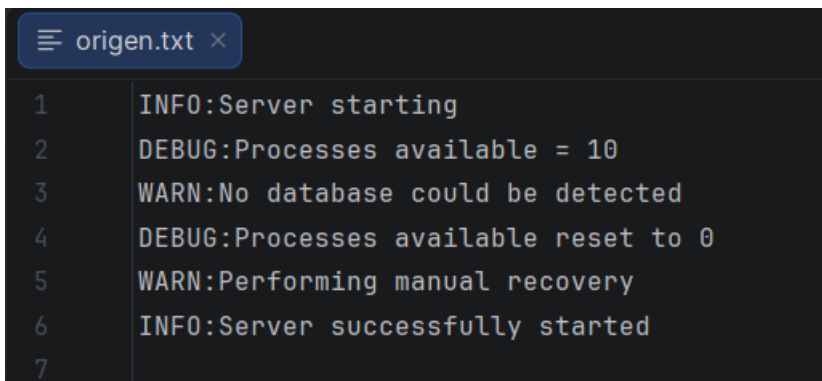
    }

}
```

El código en teoría funciona así:

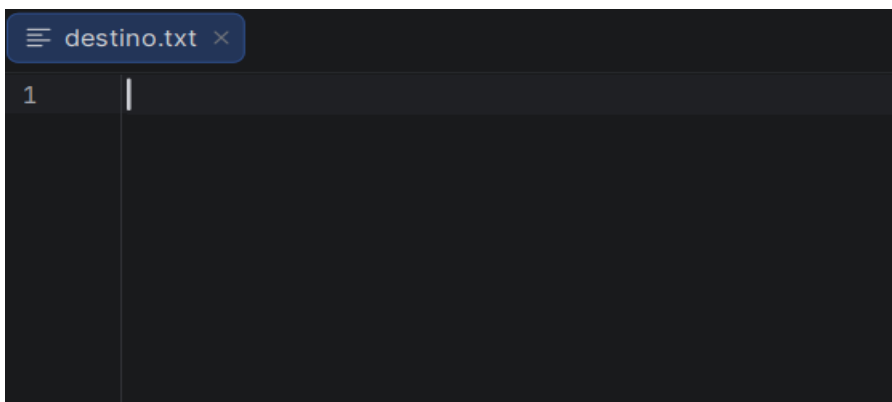
1. Usa la API moderna NIO.2 (`Paths.get(...)`) para ubicar dónde está el archivo `origen.txt` y dónde queremos crear el `destino.txt`.
2. Abre el espacio usando un bloque `try-with-resources`. Esto crea un `BufferedReader` para poder leer y un `BufferedWriter` para poder escribir.
3. Entra a un ciclo `while` donde lee una línea de texto a la vez (`reader.readLine()`).
4. Si la línea tiene texto, la escribe inmediatamente en el archivo de destino y hace un salto de línea (`writer.newLine()`). Así, línea por línea, copia todo el archivo sin saturar la memoria.

Ejecución del código:



```
origen.txt x
1  INFO:Server starting
2  DEBUG:Processes available = 10
3  WARN:No database could be detected
4  DEBUG:Processes available reset to 0
5  WARN:Performing manual recovery
6  INFO:Server successfully started
7
```

Ese es el archivo que se copiará y se pegará en `destino.txt`



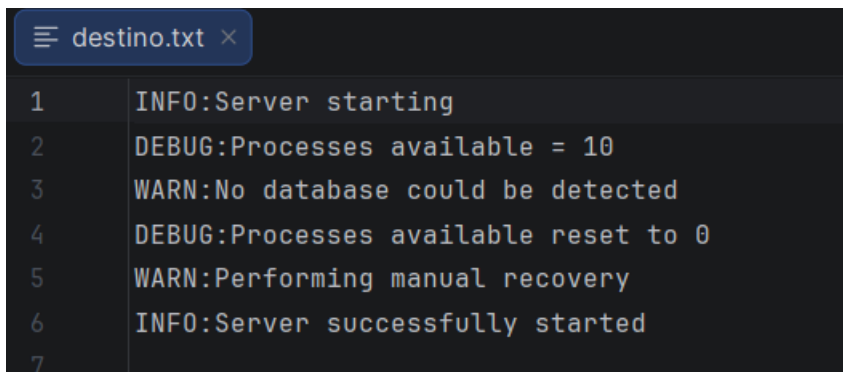
```
destino.txt x
1  |
```

El archivo destino.txt esta vacio pero una vez ejecutando el programa se copiará lo de origen.txt.



The screenshot shows the 'Run' console of an IDE. At the top, there are two tabs: 'Principal1' and 'PrincipalPath03'. The console output shows the command: `"C:\Program Files\Eclipse Adoptium\jdk-21.0.12-hotspot\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ" Listo!!!`. Below this, it says 'Process finished with exit code 0'. On the left side of the console, there is a vertical toolbar with icons for running, debugging, and other actions.

Se ejecutó correctamente.



The screenshot shows a text file named 'destino.txt' with a list of log messages. The messages are numbered 1 through 7. The log messages are: 1. INFO:Server starting, 2. DEBUG:Processes available = 10, 3. WARN:No database could be detected, 4. DEBUG:Processes available reset to 0, 5. WARN:Performing manual recovery, 6. INFO:Server successfully started, 7. (empty line).

Ahora destino.txt tiene los datos de origen.txt.

Si no usamos el try-with-resources, el sistema operativo deja el archivo "bloqueado". Nadie lo puede borrar ni modificar, y generamos una fuga de memoria o memory leak. Y si no usamos Streams, intentar leer un archivo de muchos gigas lanzará el temido error OutOfMemoryError.

Concepto 1.3: Serialización

Si intentamos guardar los datos de un sistema en puro texto, se pierde toda la magia de la Programación Orientada a Objetos. Para volver a usarlos, habría que leer el texto y armar los objetos a mano. La Serialización es la solución directa a esto ya que agarra un objeto de Java tal cual está en la memoria con sus atributos, lo detiene convirtiéndolo en un archivo binario y lo guarda. La Deserialización hace lo opuesto: lee esos bytes y revive el objeto intacto para seguir trabajando con él.

Toda esto se encuentra en el paquete `com.serializable.v2`. Funciona con tres archivos principales:

1. `Gorilla.java`: Es la clase que define al objeto. Tiene que llevar implements `Serializable` para que Java le dé permiso de ser guardado en disco.
2. `PrincipalObjectOutput.java`: Este código crea una lista de objetos `Gorilla` con datos (como Koko o Kong). Luego, abre un tubo de salida (`ObjectOutputStream`), recorre la lista con un ciclo `for` y va empujando cada objeto al archivo `gorillas.data`. Aquí los objetos quedan guardados en código binario.
3. `PrincipalObjectInput.java`: Este hace la magia inversa. Abre el archivo `gorillas.data` con un `ObjectInputStream`, va leyendo los bytes y los "castea" de regreso a objetos tipo `Gorilla` para imprimirlos en consola.


```
package src.com.serializable.v2;

import java.io.Serializable;

class Gorilla implements Serializable {

    private static final long serialVersionUID = 1L;
    private String name;
    private int age;
    private Boolean friendly;
    private transient String favoriteFood;
    private double weight = 10;

    public Gorilla(String name, int age, Boolean friendly, String
favoriteFood) {
        this.name = name;
        this.age = age;
        this.friendly = friendly;
        this.favoriteFood = favoriteFood;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }

    public Boolean getFriendly() {
        return friendly;
    }

    public void setFriendly(Boolean friendly) {
        this.friendly = friendly;
    }

    public String getFavoriteFood() {
        return favoriteFood;
    }

    public void setFavoriteFood(String favoriteFood) {
        this.favoriteFood = favoriteFood;
    }

    @Override
    public String toString() {
        return "Gorilla [name=" + name + ", age=" + age + ", friendly=" +
friendly + ", favoriteFood=" + favoriteFood
        + ", weight=" + weight + "]\n";
    }
}
```

```
package src.com.serializable.v2;

import java.io.BufferedInputStream;
import java.io.EOFException;
import java.io.File;
import java.io.FileInputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.util.ArrayList;
import java.util.List;

public class PrincipalObjectInput {

    public static void main(String[] args) throws IOException,
        ClassNotFoundException {

        String currentDir = System.getProperty("user.dir");
        File file = new File(currentDir + "/data/gorillas.data");

        List<Gorilla> gorillas = readFromFile(file);

        gorillas.forEach(System.out::println);

        System.out.println("Listo!!!");
    }

    static List<Gorilla> readFromFile(File dataFile) throws IOException,
        ClassNotFoundException {

        var gorillas = new ArrayList<Gorilla>();

        try (var in = new ObjectInputStream(
            new BufferedInputStream(
                new FileInputStream(dataFile)))) {
            while (true) {
                var object = in.readObject();
                if (object instanceof Gorilla g)
                    gorillas.add(g);
            }
        } catch (EOFException e) {
            return gorillas;
        }
    }
}
```

```
package src.com.serializable.v2;

import java.io.BufferedOutputStream;
import java.io.File;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectOutputStream;
import java.util.ArrayList;
import java.util.List;

public class PrincipalObjectOutput {

    public static void main(String[] args) throws IOException {

        String currentDir = System.getProperty("user.dir");
        File file = new File(currentDir + "/data/gorillas.data");

        // Crear una lista de 10 gorilas
        List<Gorilla> gorillas = new ArrayList<>();

        gorillas.add(new Gorilla("Koko", 12, true, "Bananas"));
        gorillas.add(new Gorilla("Kong", 25, false, "Frutas tropicales"));
        gorillas.add(new Gorilla("Bubbles", 8, true, "Manzanas"));
        gorillas.add(new Gorilla("Magilla", 15, true, "Nueces"));
        gorillas.add(new Gorilla("Harambe", 17, true, "Hojas verdes"));
        gorillas.add(new Gorilla("Enzo", 20, false, "Caña de azúcar"));
        gorillas.add(new Gorilla("Lucy", 10, true, "Bayas"));
        gorillas.add(new Gorilla("Coco", 5, true, "Mangos"));
        gorillas.add(new Gorilla("Brutus", 22, false, "Melones"));
        gorillas.add(new Gorilla("Nala", 7, true, "Plátanos"));
        gorillas.add(new Gorilla("KingKong", 20, true, "Plátanos"));

        saveToFile(gorillas, file);

        System.out.println("Listo!!!");
    }

    static void saveToFile(List<Gorilla> gorillas, File dataFile) throws
    IOException {
        try (var out = new ObjectOutputStream(
            new BufferedOutputStream(
                new FileOutputStream(dataFile)))) {

            for (Gorilla gorilla : gorillas)

                out.writeObject(gorilla);

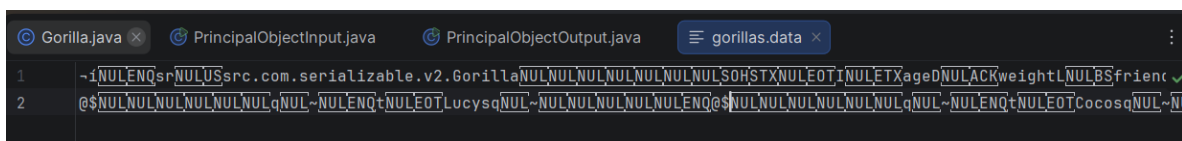
        }
    }
}
```

Esos son los fragmentos de código y a continuación se muestra el cómo funciona con la ejecución:



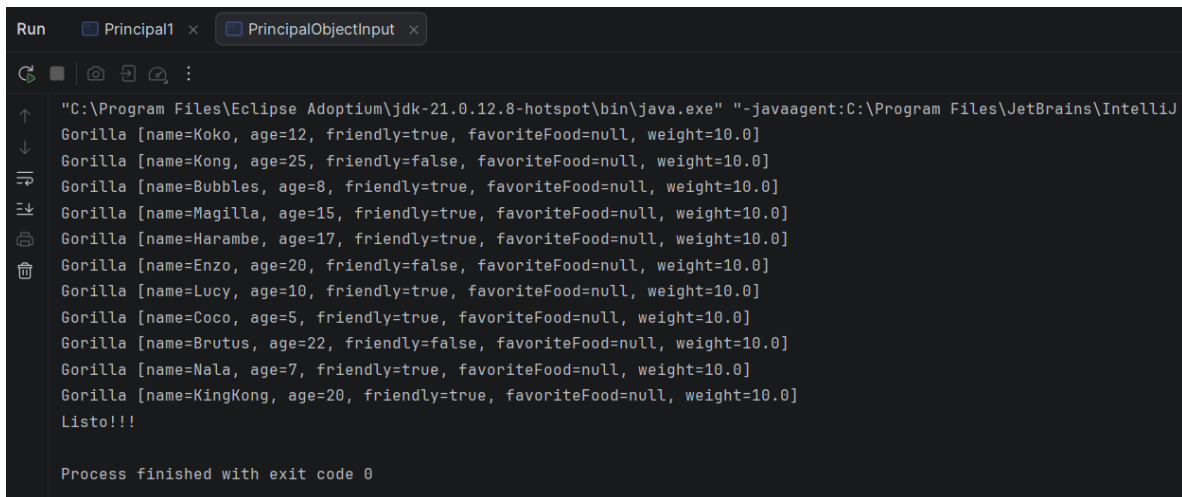
```
Run Principal1 x PrincipalObjectOutput x
"C:\Program Files\Eclipse Adoptium\jdk-21.0.12-hotspot\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ
Listo!!!
Process finished with exit code 0
```

Se ejecuta ObjectOutputStream crea un archivo .data que no se puede leer.



```
Gorilla.java x PrincipalObjectInput.java PrincipalObjectOutput.java gorillas.data x
1 ~iNULENQsrNULUSsrc.com.serializable.v2.GorillaNULNULNULNULNULNULNULSOHSTXNULEOTiNULETXagedNULACKweightLNULBSfrienc ✓
2 @$NULNULNULNULNULNULqNUL~NULENQ+NULEOTLucysqNUL~NULNULNULNULNULNULqNUL~NULENQ+NULEOTCocosqNUL~NUL
```

Aquí es donde entra el ObjectInputStream que es quien lo coloca a la inversa y muestra los datos.



```
Run Principal1 x PrincipalObjectInput x
"C:\Program Files\Eclipse Adoptium\jdk-21.0.12-hotspot\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ
Gorilla [name=Koko, age=12, friendly=true, favoriteFood=null, weight=10.0]
Gorilla [name=Kong, age=25, friendly=false, favoriteFood=null, weight=10.0]
Gorilla [name=Bubbles, age=8, friendly=true, favoriteFood=null, weight=10.0]
Gorilla [name=Magilla, age=15, friendly=true, favoriteFood=null, weight=10.0]
Gorilla [name=Harambe, age=17, friendly=true, favoriteFood=null, weight=10.0]
Gorilla [name=Enzo, age=20, friendly=false, favoriteFood=null, weight=10.0]
Gorilla [name=Lucy, age=10, friendly=true, favoriteFood=null, weight=10.0]
Gorilla [name=Coco, age=5, friendly=true, favoriteFood=null, weight=10.0]
Gorilla [name=Brutus, age=22, friendly=false, favoriteFood=null, weight=10.0]
Gorilla [name=Nala, age=7, friendly=true, favoriteFood=null, weight=10.0]
Gorilla [name=KingKong, age=20, friendly=true, favoriteFood=null, weight=10.0]
Listo!!!
Process finished with exit code 0
```

Si intentas obligar a un objeto a guardarse en un archivo binario (usando `ObjectOutputStream`), pero la clase no tiene el `implements Serializable`, Java bloquea el proceso por seguridad y el programa hace crash con el error `NotSerializableException`.

- ¿Para qué sirve el `serialVersionUID` (ej. 1L)?

Es el control de versiones. Si el código de la clase `Gorilla` llega a modificarse en el futuro se va agregando un nuevo campo, la versión cambia. Cuando Java intenta leer el archivo viejo, se da cuenta de que el ID de versión ya no cuadra con el nuevo código y lanza un `InvalidClassException`. Esto evita que el sistema cargue información descuadrada.

- ¿Qué pasa con el atributo marcado como `transient`?

En la clase `Gorilla`, el campo `favoriteFood` tiene la etiqueta `transient`. Esto le dice a Java que salte este dato, no se guardará en el disco. Por eso, al correr la lectura del archivo (`PrincipalObjectInput`), todos los gorilas regresan a la vida, pero su comida favorita aparece en `null`. Sirve para proteger información sensible, como contraseñas o tokens.